Name :- Viraj Khade

Reg No :- RA2311003010919

Subject :- NLP

RNN Code

```
import numpy as np
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, SimpleRNN, Dense
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences

# Sample dataset
texts = ["I love this", "I hate this", "This is amazing", "This is
bad"]
labels = [1, 0, 1, 0]

# Tokenization
tokenizer = Tokenizer()
tokenizer.fit_on_texts(texts)
sequences = tokenizer.texts_to_sequences(texts)

X = pad_sequences(sequences, maxlen=3)
y = np.array(labels)

# Model
model = Sequential([
    Embedding(input_dim=50, output_dim=8, input_length=3),
    SimpleRNN(16),
    Dense(1, activation='sigmoid')
])

model.compile(optimizer='adam', loss='binary_crossentropy',
metrics=['accuracy'])
model.fit(X, y, epochs=20, verbose=0)

# Test input
test_text = ["I love it"]
test_seq = pad_sequences(tokenizer.texts_to_sequences(test_text),
maxlen=3)
```

```
prediction = model.predict(test_seq)
print("Input:", test_text)
print("Output (RNN prediction):", prediction)
```

Output

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 249ms/step
Input: ['I love it']
Output (RNN prediction): [[0.47637847]]
```

LSTM Code

```
import numpy as np
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, LSTM, Dense
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences

# Sample dataset (slightly longer sentences to show memory)
texts = [
    "I really love this product",
    "I really hate this product",
    "This is not good at all",
    "This is absolutely amazing"
]

labels = [1, 0, 0, 1]  # 1 = positive, 0 = negative

# Tokenization
tokenizer = Tokenizer()
tokenizer.fit_on_texts(texts)
sequences = tokenizer.texts_to_sequences(texts)

# Padding (important for LSTM)
X = pad_sequences(sequences, maxlen=6)
y = np.array(labels)

# LSTM Model
model = Sequential([
    Embedding(input_dim=100, output_dim=16, input_length=6),
    LSTM(32),   # <-- key difference from RNN
    Dense(1, activation='sigmoid')
])
```

```
model.compile(optimizer='adam',
              loss='binary_crossentropy',
              metrics=['accuracy'])

# Train
model.fit(X, y, epochs=25, verbose=0)

# Test input
test_text = ["I do not love this"]
test_seq = pad_sequences(tokenizer.texts_to_sequences(test_text),
maxlen=6)

prediction = model.predict(test_seq)

print("Input:", test_text)
print("Output (LSTM prediction):", prediction)
```

Output

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 183ms/step
Input: ['I do not love this']
Output (LSTM prediction): [[0.5121883]]
```

BERT Code

```
from transformers import pipeline

# Load pretrained model
classifier = pipeline("sentiment-analysis")

# Input
text = "I love using BERT models!"

# Output
result = classifier(text)

print("Input:", text)
print("Output (BERT prediction):", result)
```

Output

```
No model was supplied, defaulted to distilbert/distilbert-base-uncased-finetuned-sst-2-english and revis
Using a pipeline without specifying a model name and revision in production is not recommended.
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:93: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/s
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
  warnings.warn(
config.json: 100%  629/629 [00:00<00:00, 44.9kB/s]
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Ple
model.safetensors: 100%  268M/268M [00:03<00:00, 149MB/s]
Loading weights: 100%  104/104 [00:00<00:00, 451.38it/s, Materializing param=pre_classifier.weight]
tokenizer_config.json: 100%  48.0/48.0 [00:00<00:00, 4.06kB/s]
vocab.txt:  232k/? [00:00<00:00, 6.58MB/s]
Input: I love using BERT models!
Output (BERT prediction): [{'label': 'POSITIVE', 'score': 0.99933260679245}]
```

RoBERTa Code

```python
from transformers import pipeline

# Load RoBERTa sentiment model
classifier = pipeline(
    "sentiment-analysis",
    model="cardiffnlp/twitter-roberta-base-sentiment"
)

# Input
text = "I do not like this product at all, it's terrible!"

# Output
result = classifier(text)

print("Input:", text)
print("Output (RoBERTa prediction):", result)
```

Output

```
config.json: 100%  747/747 [00:00<00:00, 19.7kB/s]
pytorch_model.bin: 100%  499M/499M [00:05<00:00, 150MB/s]
Loading weights: 100%  201/201 [00:00<00:00, 498.61it/s, Materializing param=roberta.encoder.layer.11.output.dense.weight]
model.safetensors:  72%  359M/499M [00:04<00:01, 123MB/s]
RobertaForSequenceClassification LOAD REPORT from: cardiffnlp/twitter-roberta-base-sentiment
Key                            | Status     |  |
-------------------------------+------------+--+-
roberta.embeddings.position_ids | UNEXPECTED |  |

Notes:
- UNEXPECTED    :can be ignored when loading from different task/architecture; not ok if you expect ider
vocab.json:  899k/? [00:00<00:00, 7.41MB/s]
merges.txt:  456k/? [00:00<00:00, 9.75MB/s]
special_tokens_map.json: 100%  150/150 [00:00<00:00, 3.08kB/s]
Input: I do not like this product at all, it's terrible!
Output (RoBERTa prediction): [{'label': 'LABEL_0', 'score': 0.9812012314796448}]
```